

QUIÉN	RAMA	BLOQUE	ARCHIVOS QUE LE PERTENECEN – NADIE MÁS LOS TOCA
Sergio	main	spec	specs/002-*/**, .specify/feature.json
Mateo	feature/hu-ui	A · pantallas	app/**/page.jsx, components/**, hooks/**
Johan	feature/hu-api	B · API	app/api/**/route.js y sus .test.js
Tomás	feature/hu-datos	C · datos	test/** – dobles de Mongo y de HTTP, fixtures
Santiago	main	merge	scripts/**

0

Los cinco, antes de entrar

cada uno en su máquina

```
-20
min
cd ~/Documents/web/speckit-ai-generator
git checkout main && git pull origin main && npm install
npm test # 0 failed
npx -y react-doctor@latest . --score # ≥ 95
npm run smoke https://ia-generator-openspec.vercel.app
```

Puerta: si el smoke sale rojo, lo desplegado es viejo y la demo arranca rota. Se redespiega `main` antes de entrar, no en clase.

1

Sergio — la Historia entra como feature nueva

rama main

≈4
min

`/speckit-specify` historia literal que dictó el profesor

Es una feature nueva encima del producto que ya existe en `specs/001-ai-media-generator/spec.md`. NO edites esa carpeta: leela para no contradecirla y escribe todo en la nueva. NO crees `checklists/`.

`/speckit-plan` Reutiliza la arquitectura de `specs/001-ai-media-generator/plan.md`, no inventes otra. Fija el contrato HTTP y el reparto por propiedad de archivos en tres bloques disjuntos: A pantallas (`app/**/page.jsx`, `components/**`, `hooks/**`), B API (`app/api/**/route.js`), C datos (`test/**`). NO edites `specs/001-ai-media-generator/`.

`/speckit-tasks` Agrupa por los tres bloques que `plan.md` ya define, no por User Story. Encabezados exactos `### Bloque A -`, `### Bloque B -`, `### Bloque C -`: nunca la palabra "Fase". Cada tarea lleva `[US1-N]`, es `test-first`, y manda hacer `commit` de la prueba en rojo con mensaje `"test: ..."` antes del código. Ningun bloque importa un símbolo nuevo de un archivo de otro bloque. NO edites `specs/001-ai-media-generator/`.

```
cat .specify/feature.json # specs/002-... - si dice 001, no sirvió
git status --short # specs/001 intacto · sin carpeta checklists/
git add specs/ .specify/feature.json
git commit -m "HU en vivo: feature 002" && git push origin main
```

En voz alta: «bloque A Mateo, bloque B Johan, bloque C Tomás — arranquen».

2

Mateo · Johan · Tomás — en paralelo

cada uno su columna, de arriba abajo

≈6
min

MATEO · BLOQUE A
git checkout main
git pull origin main
cat .specify/feature.json # 002
git checkout -b feature/hu-ui

/speckit-implement bloque A
solo ese bloque. no toques
ningun archivo de config.

npm test
npx -y react-doctor@latest . --score
git add -A
git commit -m "feat: HU pantallas"
git push -u origin feature/hu-ui

JOHAN · BLOQUE B
git checkout main
git pull origin main
cat .specify/feature.json # 002
git checkout -b feature/hu-api

/speckit-implement bloque B
solo ese bloque. no toques
ningun archivo de config.

npm test
npx -y react-doctor@latest . --score
git add -A
git commit -m "feat: HU API"
git push -u origin feature/hu-api

TOMÁS · BLOQUE C
git checkout main
git pull origin main
cat .specify/feature.json # 002
git checkout -b feature/hu-datos

/speckit-implement bloque C
solo ese bloque. no toques
ningun archivo de config.

npm test
npx -y react-doctor@latest . --score
git add -A
git commit -m "feat: HU datos"
git push -u origin feature/hu-datos

Puerta: rojo en `npm test` o score bajo 95 es **no-push**. «Archivo de config» = `.gitignore`, `eslint.config.mjs`, `next.config.mjs`, `package.json`, `vitest.config.mjs`. Anuncia tu push en voz alta.

3

Santiago — junta y despliega

solo cuando los tres hayan anunciado

```
≈2
min
git checkout main && git pull origin main && git fetch origin
git merge --no-ff origin/feature/hu-datos # C primero, luego B, luego A
git merge --no-ff origin/feature/hu-api
git merge --no-ff origin/feature/hu-ui
npm run lint && npm test && npx -y react-doctor@latest . --score && npm run build
git push origin main # las feature/* tambien lanzan preview: ignoralos
git log --graph --oneline -12 # proyecta esto mientras Vercel construye main
```

Un merge por línea, esperando a que termine. Si uno se enreda: `git merge --abort` — esa rama salió de su bloque.

4

Todos — validan sobre el despliegue

con datos móviles, no con el wifi del salón

```
≈1
min
npm run smoke https://ia-generator-openspec.vercel.app # tras el check verde
npm test -- -t "US1-3" # unas passed, el resto skipped
```

Trazabilidad, si preguntan: `tasks.md` en su bloque → la etiqueta `[US1-3]` → esa prueba → el `Given/When/Then` número 3 en `spec.md` → `git log`, donde el `commit` `test:` va antes del código.

OCHO COSAS QUE ROMPEN ESTO – SALIERON EN EL ENSAYO			
01	Nunca la palabra «fase». <code>specs/001/tasks.md</code> tiene una «Fase 1» que corre <code>create-next-app</code> y re-andamia el proyecto. Por eso: bloques .	04	El agente «verifica la config». Toca <code>.gitignore</code> y <code>eslint.config.mjs</code> ; si los tres lo hacen, el merge explota.
02	<code>git pull</code> antes de implementar. Sin el <code>.specify/feature.json</code> nuevo, tu agente lee el spec viejo y no construye nada.	05	El doble de <code>fetch</code> responde por orden. La lectura al montar se come la primera respuesta y tumba cinco pruebas viejas. Que responda por método .
03	<code>checklists/</code> congela a los tres. Una casilla vacía y <code>/speckit-implement</code> se detiene a preguntar «¿sigo? (yes/no)».	06	Score bajo 95 no despliega. <code>fetch</code> en un hook, no en <code>useEffect</code> de <code>page.jsx</code> ; <code>modal <dialog></code> ; <code>flatMap</code> , no <code>.filter().map()</code> .
07	Nombres accesibles que chocan. Dos <code>alt</code> parecidos, o un botón que toma su nombre del <code>alt</code> , rompen <code>getByRole</code> viejo. <code>aria-label</code> explícito.		
08	El helper de pruebas no exporta lo nuevo. <code>load()</code> devuelve <code>{ POST, ...mocks }</code> : sin agregarle <code>GET</code> falla con <code>GET is not a function</code> .		